**Year 4 Project**
[CSC1097]

# Tag Battle Royale

[Technical Specification]


| | | | |
|---|---|---|---|
| Project Team: | Luan Slankster | luan.slankster2 @mail.dcu.ie | 22406536 |
| | Michael Cojocaru | michael.cojocaru2 @mail.dcu.ie | 22380876 |
| Supervisor: | Darragh O'Brien | darragh.obrien @dcu.ie | |


**1st May 2026**

**Table of contents**


**1. Introduction....4**
1.1 Motivation....4
1.2 Place in the Market....4
**2. Research....5**
2.1 Market Research....5
2.2 Application Development Framework....6
2.3 Connectivity....6
2.4 Scatternet....7
**3. High-Level Design....8**
3.1 Main Menu....9
3.2 Map Editor....9
3.3 Face Detector....9
3.4 Game Phases....9
3.5 Game Screen....10
3.6 Active Phase....10
3.7 Finished Phase....10
3.8 Sleep and Interruptions....11
3.9 Offline Connectivity....11
3.10 Online Connectivity....12
3.11 Replay Viewer....12
3.12 High-Level Diagrams....13
3.12.1 State diagram for an Online Game:....13
3.12.2 State diagram for an Offline Game:....13
3.12.3 Sequence diagram for compiling a replay in an Offline Game:....14
**4. Implementation....15**
4.1 Code Implementation....15
4.1.1 Map editor....15
4.1.2 Face Detector....15
4.1.3 Game Screen....19
4.1.4 Offline Connectivity....20
4.1.5 Online Connectivity....22
4.1.6 Location Service....24
4.1.7 Replays....24
4.1.8 File and Session Storage map_file_handler.dart and replay_file_handler.dart....25
4.1.9 Google Maps....26
4.2 Project Implementation....26
4.2.1 Agile....26
4.2.2 Continuous Integration (CI)....27
**5. Problems Solved....27**

5.1 Secure API Access..... 27
5.2 Displaying Users..... 27
5.3 Face Detection..... 27
5.4 Offline Connectivity: Scatternet Pseudo-host Allocation..... 28
5.5 Offline Connectivity: Chunking..... 28
5.6 Offline Connectivity: Endpoint Names..... 28
5.7 Resuming Games..... 29
5.8 Disposability..... 29
5.9 Replay Construction..... 30
**6. Future Work..... 30**
6.1 General Feedback..... 30
6.2 Additional Features..... 31
- 6.2.1 Automatic Map Features..... 31
- 6.2.2 Additional Game Modes/Features..... 32
**7. Conclusion..... 32**
**8. References..... 33**

# 1. Introduction

Tag Battle Royale is an android application that facilitates a traditional game of Tag but on a much larger scale. The gameplay can be broadly described as this: A map is used to create borders in which the game is played. The map contains zones of play which are closed periodically throughout the game to restrict where players can move and bring them closer together. There are two groups of players, Runners and Chasers. The objective of the Chaser is to tag all Runners which converts their role to Chaser. The last Runner to be tagged is deemed the winner.

### 1.1 Motivation

This game was originally played by one of the members of this project through rudimentary means. Messaging apps were used to communicate information such as zone closings and image editing applications were used to create the map and its zones. The game was extremely fun to play but the implementation had many issues such as unreliable messaging and ambiguous zone edges. Our app Tag Battle Royale was designed to be a single application that can facilitate the whole game and provide extra features.

### 1.2 Place in the Market

The original basic version of this game was played due to the lack of existing applications that address the desire for a large-scale outdoor type of game. Organising games like these can be a hassle, so one would think the desire for this type of application would be high. Unfortunately, there is very little attention given to this area by other applications out there. Tagtile was the most recent application we found to modernise tag in a way similar to what our project has done, but this application never became publicly available and stopped receiving updates in 2012. This could be due to lack of interest but also partly due to lack of network availability or clunkiness. In 2012, for example, mobile networks like 5G were not readily available for use throughout the world and various offline network capabilities such as Nearby Devices API were not developed yet either.

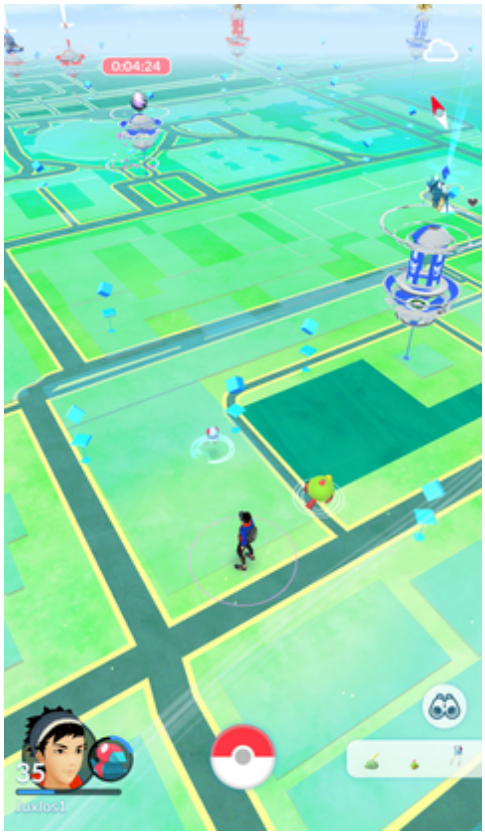
Our testers say Tag Battle Royale's main gameplay loop is fun and allows for use in situations such as enterprise team building exercises. As more and more modern entertainment encourages people to stay indoors, Tag Battle Royale presents itself as an exciting alternative.

# 2. Research

To tackle this project, we had to make a number of choices on the use of our technologies. Conducting research would be beneficial in creating our application with the right tools and knowing what goals to strive for in the formation of the project.

### 2.1 Market Research

We looked into the market to conduct research on popular applications that use multiplayer gameplay but are meant to be played physically. Applications like Niantic's *Pokemon GO (2016)* that leverage a player's real world position as a part of a larger, long-time, ongoing game where every player is inside of a shared world.




***Pokemon Go (2016)***
*Niantic*

Other applications like Ellen Digital Ventures's and Gasket Games's *Heads Up (2013)* provide an entertaining in person multiplayer experience on a smaller scale, organised within an already grouped set of people. As mentioned previously, we couldn't find modern popular applications that partly mixed the two of these types of games together, group-based play in a larger environment.

The key to the success of a game in the digital market is complex and based on many unknown factors. We can't say for sure what made games popular or unpopular but we can do some analysis to create a hypothesis. *Pokemon GO* is clunky to play, it requires users to move around in the real world and then, when arriving at a location, look at their phone for some time, but it can get away with this due to the type of game it is. *Pokemon GO* is not a game played in rounds in groups of people, it's meant to be an ongoing continual experience. On the other hand, group-based short round games like *Heads Up* are made to be accessible and easy to use. More complex party games on the Google Play Store, something like InnerSloth's *Among Us (2018)*, don't use physical elements and are played entirely on devices.

Tag Battle Royale is in a way, our attempt to fuse the categories of largescale, physical and group-based play. Our game won't be a continual experience, but what we can take away is that we need the experience to be as accessible and easy to use as possible to potentially see a chance at success in a larger market.

### 2.2 Application Development Framework

After careful consideration, we chose to develop Tag Battle Royale with Flutter, developed by Google. Flutter provides an open-source framework for building mobile applications. It allows for fast, easy and elegant development with many existing libraries and packages. It was especially attractive with our goal of accessible UI and its compatibility with the Nearby Connections API which we will speak on later.

Flutter puts every UI component into a “widget”. Flutter’s use of widgets gives a single, consistent model to work with, complex interfaces are built using small widgets together like building blocks. Everything is manageable and easily broken into pieces, which would be beneficial for us trying to craft an accessible application. Customisation is flexible, because components are not tied to a base class, even low-level properties such as padding can be customised per widget.

Widgets are declared in a tree. If an error occurs or part of the tree is changed, the whole application does not crash. The tree acts as layers that can be rebuilt and cut off from the rest. The trade-off is the code can be quite verbose, requiring large amounts of nesting.

### 2.3 Connectivity

We knew we wanted a way to play a stripped version of the game offline and a way to play that was enhanced by the use of network connectivity, due to the game being played outdoors and stable network connectivity not being guaranteed. We did not put a lot of time looking into how the online version was to be conceived. Both of us had experience creating servers in previous projects and we knew a simple NodeJS server with the use of WebSockets would always be flexible enough to do the trick for smaller student projects like these. We simply wanted to have the online version working while we put thought into the functionality of offline play.

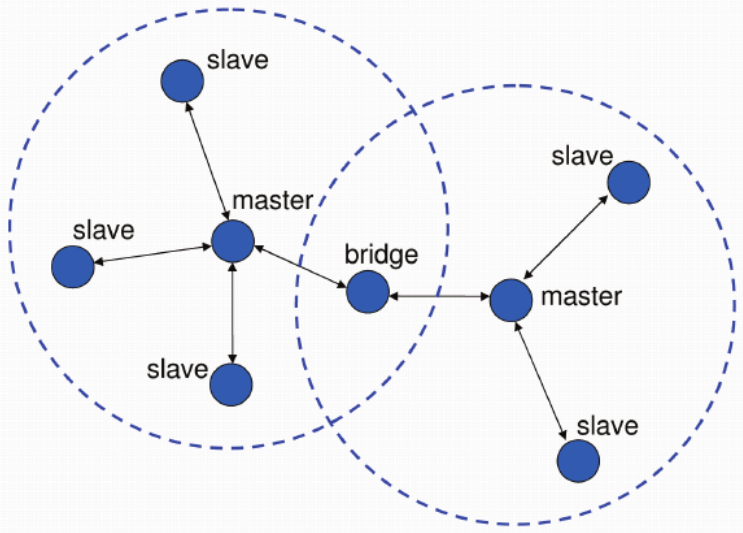
At first, we looked at the possibility of using a Bluetooth connection. Bluetooth was appealing to use because of its universal adoption in all mobile devices, but Bluetooth is an old service that can be difficult to develop for and manage in modern applications. We seldom found good libraries or documentation on how to effectively utilize it in our proposed application development framework. NFC connectivity was also looked at for the same reason. NFC connectivity could be useful as a simple quick way to distribute information to a group such as the map and zones the game will take place in, allowing the game to start as quickly as possible. But we wanted to create a virtual lobby before the game began so we ruled it out.

Finally, we came across the Nearby Connections API by Google. This service leverages Wi-Fi Direct, Bluetooth Low Energy (BLE) and Bluetooth connectivity to create high bandwidth connections between devices. The service was made accessible and easy to use and also integrated with the application development framework we were looking at, also developed by Google.

### 2.4 Scatternet

A problem apparent in the use of offline connectivity is its limit in the amount of devices capable of being connected at once. Reportedly, Nearby Connections API starts to undergo disconnections and slowness at higher amounts of concurrent connections. We were unable to find concrete information on the limit of the amount of connections able to be held by a single device at once using Nearby Connections, but we learnt that the technologies it leverages puts it around seven. We could learn more through testing.

The solution proposed in our functional specification was to create a Scatternet. Scatternet is an old concept, and we found game projects from as early as even 2007 [1] utilizing the idea with Bluetooth connectivity. Nevertheless, we thought it was an obvious and novel way of tackling the problem, and would further our understanding of how multiplayer networking could be done.

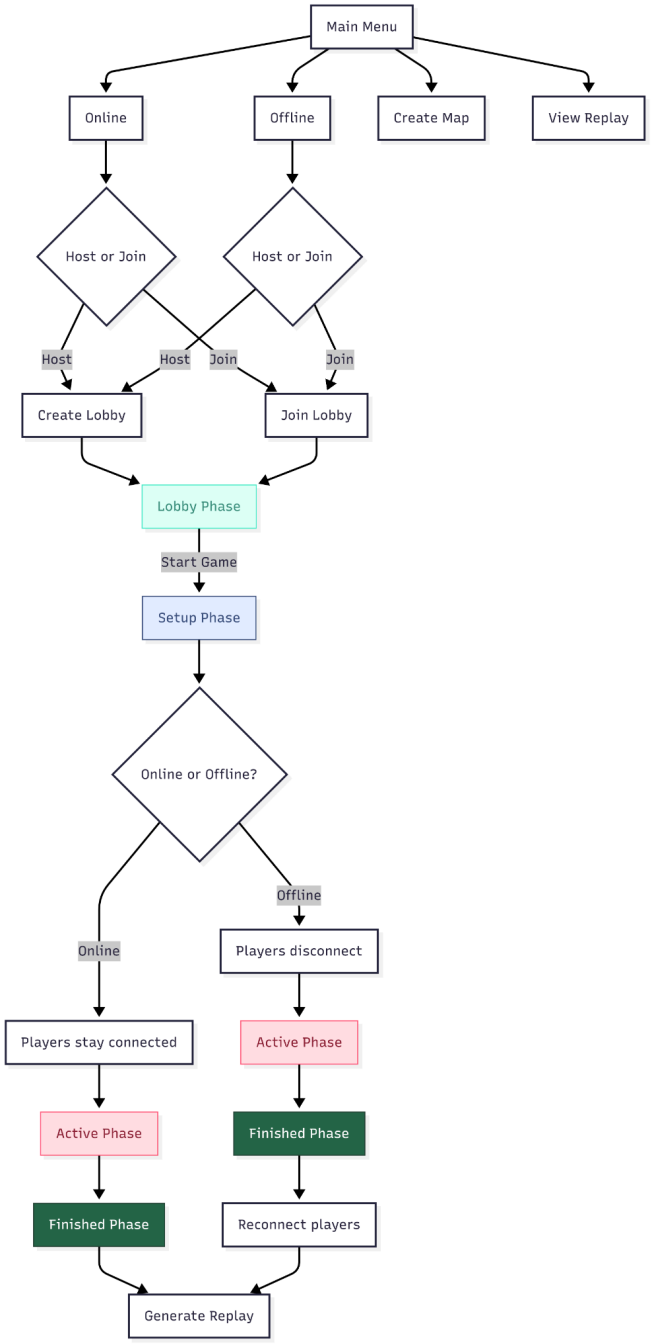



*Scatternet Example [2]*

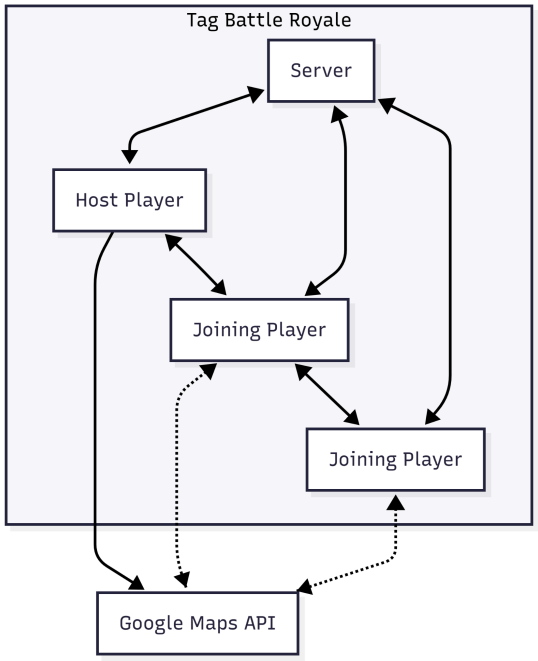
# 3. High-Level Design

Our design did not drift too far from what was described in the functional specification. We've created a new system flowchart to illustrate how a user interacts with the app. Our system architecture diagram has the same idea of the original diagrams, this time combined into one more explanatory diagram.

**Tag Battle Royale Flowchart:**




**Tag Battle Royale System Architecture:**



### 3.1 Main Menu

On a fresh launch of the app, the main menu screen presents the user with four options: Online Play, Offline Play, the Map Editor and Replay Viewer. The four options are split into two pairs with the use of colour coding. Both Play options present the user with the option to either host or join a lobby. Whatever Play option is chosen, the user is brought to the same lobby screen which also accommodates the full game. Both the Map Editor and Replay Viewer will bring the user to similar list screens. We wanted the layout of the app to be coherent and easily understood.

### 3.2 Map Editor

Before a game begins, a map needs to be made and selected. The Map Editor screen is accessible through the main menu screen but also by the host of a game in a lobby, during the lobby phase. Here, a list of maps are presented and individual maps can be viewed and selected if in game. The screen contains the option to create a new map if the user is connected to the internet.

When creating a new map, the user is first shown a map embed at their current location. The user can navigate to where they plan to play the game and capture this area as an image. This image can then be drawn on with the tools provided by the image editor. These tools are the pen tool, which allows the user to continuously draw where their finger specifies, the straight-line tool, which allows the user to specify the start and end of a line, and the eraser tool, which allows the user to remove a line. The image editor also provides an undo and redo button to undo and redo drawing actions. When a line is drawn, the lines on the map will be used to detect the faces drawn by the user. The user can then name the map, name the zones drawn and finally save the map.

### 3.3 Face Detector

To find the zones drawn by the user we implemented a face detector. The face detector takes the strokes drawn by the user, simplifies the strokes, uses the processed strokes to create segments from consecutive points in a stroke, finds all the intersections between the segments, generates a planar graph using the segments and intersections and finally traces the faces in that planar graph.

### 3.4 Game Phases

A game is hosted by a player who will choose the map and order of zones to play on, configure the rules of the game, and choose starting roles. There are four phases internally which comprise the flow of playing Tag Battle Royale:
Lobby Phase -> Setup Phase -> Active Phase -> Finished Phase.

- Lobby Phase: The host sets up the game to be played, chooses rules and the map. New users that join the game must be accepted by the host before gaining access to game information and being included in the Setup Phase
- Setup Phase: The host chooses to start the game, the map, zones and their closure information, grace period time and game starting time are sent to all players. The game

only begins once all players have confirmed they've received and processed the required information. This is because it may take some time for the large information to be sent and processed.
- Active Phase: The game of tag is actively played.
- Finished Phase: The game has ended, players will begin automatically trying to send their replay files to the host to be assembled together into a single file.

**3.5 Game Screen**

The game screen is used for online and offline play. The game is played here through all phases of the game, we do not move to a new page (in the metaphorical sense, because everything in Flutter is a widget). This keeps the UI layout consistent. The game screen is made up of three tabs.

- The map tab, which the host uses to choose a map, which is then converted to where players check the map and zones during the active and finished phases.
- The chat tab, which can be used to chat to other players during the lobby phase and access different chatrooms during active phase if online.
- The game tab, which holds important components to the game in sub tabs, such as game rules that can be configured by the host and player list.

**3.6 Active Phase**

In the active phase, the goal of Chasers is to tag all Runners. The map tab shows a player's location, all the zones, the player's current zone, and the time until the next zone closes. Runners are unaware of the order in which zones close, Chasers are aware and can use the information for a strategic advantage. If a Runner is caught by a Chaser, they must switch their roles on the app to gain the benefits of a Chaser.

The game was originally played with the faith that Runners outside of open zones would leave on their own, with the app, we are capable of enforcing rules. When played online, Runners outside of open zones are displayed on the map to Chasers and Chasers are able to view each other on the map, for better coordination. Players disconnected from the server are displayed with "last seen Xs ago". Because the players are continuously connected to a server, we can make use of role specific chatrooms during play too. Additionally we have the option to have automatic conversions to the Chaser role if a Runner stays outside of an open zone for too long, offline or online. Unfortunately our hands are a little bit tied in how this rule can be enforced. Internet and GPS connection can be unreliable, we put careful consideration into how a player would reconnect/continue to an ongoing match if disconnected or the app was closed, online and offline. Punishing players for not remaining in open zones could lead to unfair role conversions and will lead to increased use of a player's phone, so we made the time until conversion long, five minutes by default, and made it an optional component of the game.

**3.7 Finished Phase**

In online mode, the game ends once all Runners have had their roles converted to Chasers, or if the host chooses to manually end early (player dropped out, etc.). In offline mode, there is no

server to detect the roles of players, which means players need to manually select to end the game on their client.

Throughout the game, whenever a player retrieves location data from their GPS stream, a timestamped frame of that data is saved within their device. Once in the Finished Phase, these sets of frames are retrieved by the host, assembled into full replays and distributed back out to all clients. Additionally, a leaderboard is formed in the players sub tab of the game tab that displays the order in which players were caught and the times they were caught.

**3.8 Sleep and Interruptions**

This aspect of the app design was important. Tag Battle Royale is a game played outside for long periods of time and the majority of the game is typically meant to be played with the device's screen turned off. We made sure that the game is kept as a *foreground service* so that the device's CPU isn't put to sleep during gameplay and the app's services keep running. This means updates would keep being sent and retrieved to and from the server in online mode and replays data would continuously keep being recorded. Additionally, this is helpful because the GPS service takes some time to start up. If the device was put to sleep and turned back on again, there would be a noticeable delay until GPS updates.

However, in the case that the user is ever disconnected from the server in online mode or the app is ever fully closed for whatever reason, which is a not low possibility in the context of how the game is played, there are some design choices we have made. In online mode, the server still retains the object of a player even if they have disconnected. In the event of online mode clients losing connection to the server, or the GPS stream dropping in either mode, these services will repeatedly try to restart in intervals. The game data is continuously saved as a session to the device. When a user enters the main menu, potentially after the app closes during a game, and a previous session is available in the device's memory, the user will be presented with the option to rejoin their online game or continue their offline game. These "reconnections" simply restore the previous state of the game screen with the session information, including map data and previous replay data.

Additionally, when players disconnect and stop sending location updates, their previous location is still displayed with a timestamp of when that location was broadcast. This ensures that players are not misinformed about older location updates.

**3.9 Offline Connectivity**

The offline base functionality was the first to be worked on because of its complexity. A Scatternet is a mesh network where once the host of the network reaches its capacity of connected devices, it stops advertising and designates another device to be a pseudo-host. This pseudo-host would advertise itself and bring more devices into the network, acting as a relay between the host and the next set of devices to join. Our version of the Scatternet has a host at the top and no interconnectivity of nodes in the network, effectively making it a tree. After testing, we decided on having the max child connections for host or pseudo-host be three (not including a pseudo-host's parent).

Devices advertising in the tree identify as the name of the game lobby. Communication between devices in the network was achieved using text command protocol. Each message is prefixed with identifiers that tell devices what they do. Messages are flooded throughout the tree or can be relayed between devices to reach specific devices like the host.

### 3.10 Online Connectivity

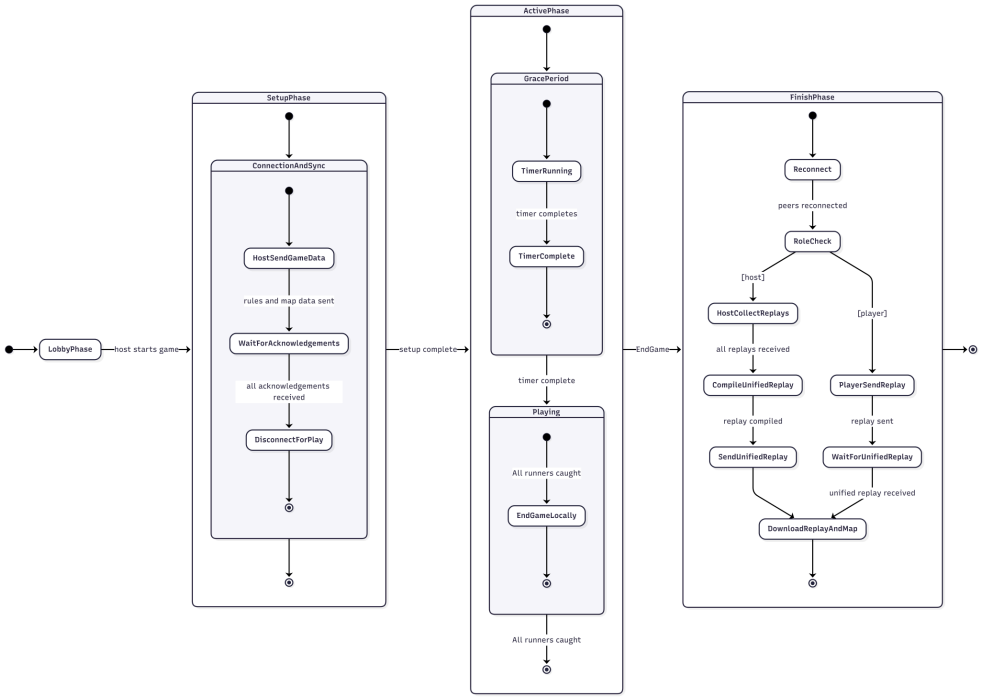
Online functionality is done using a NodeJS server with sockets, using the library Socket.IO. The server is containerised using Docker and can be uploaded to a server hosting site. This project uses the platform Fly.io and includes the necessary files to support it there. Game and player objects are stored on the server, connected sockets are tied to player objects, and player objects are tied to games. The server can and does broadcast to specific sockets by using the data in player objects. To connect to a game requires you to input the randomly generated game code in the main menu.

### 3.11 Replay Viewer

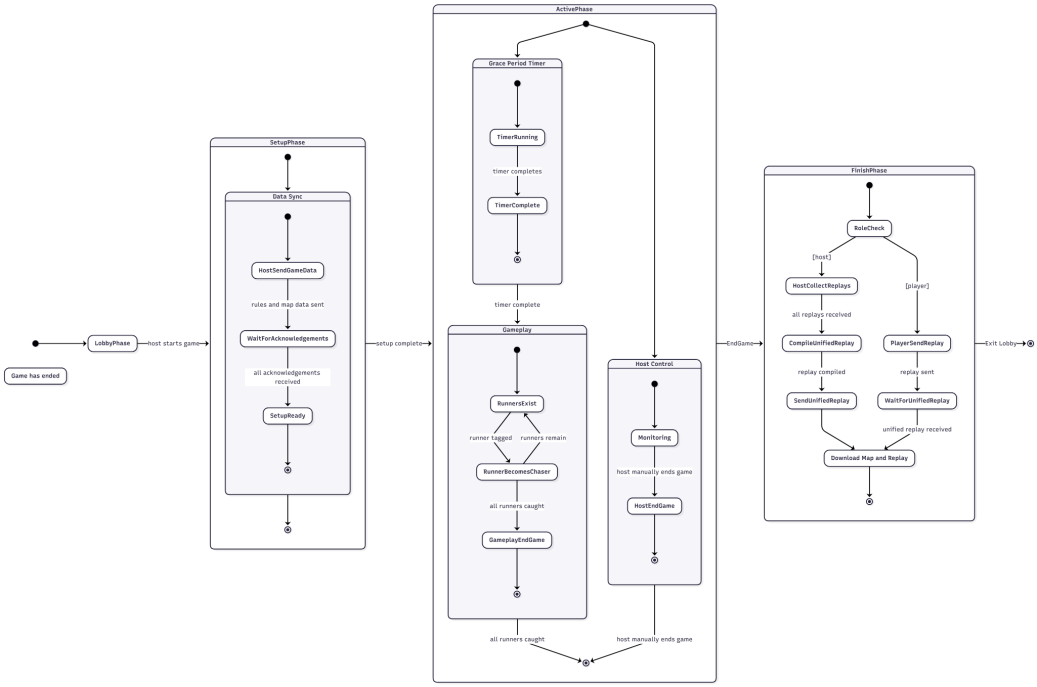
The replay viewer allows users to watch the games they've played in a video-like format. The data the replay viewer uses is captured during the online and offline games. The replay file contains the position of all the players at the different times during the game, their role and the zone closings. Using that information we display where each user is at any point during the match, what zones are closed and what role each player has. The user can press the play button to start the "video" and watch the players move around and get caught in real time. It is important to note that this isn't an actual video and that the replay information is used to render each player and their data in realtime. This allows us to provide options like only displaying certain players or hiding the nameplates during the replay.

# 3.12 High-Level Diagrams

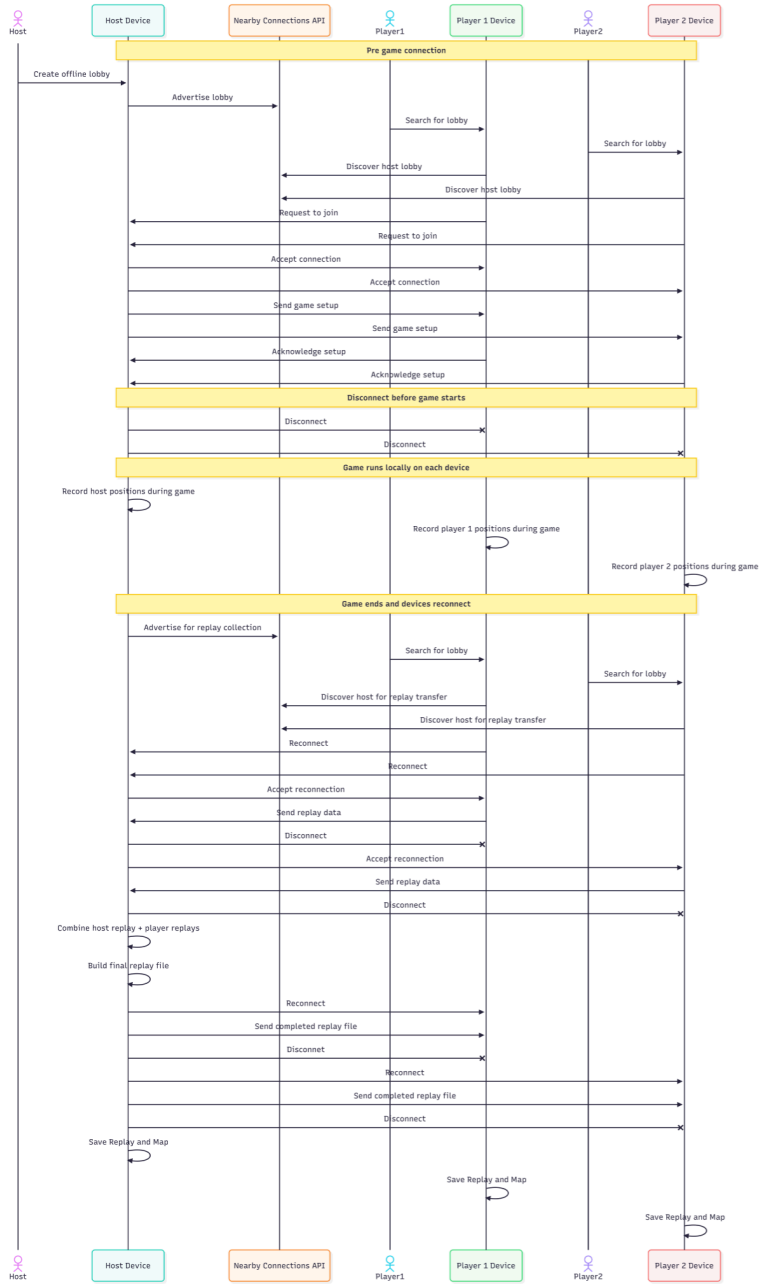
## 3.12.1 State diagram for an Online Game:




## 3.12.2 State diagram for an Offline Game:



### 3.12.3 Sequence diagram for compiling a replay in an Offline Game:



# 4. Implementation

## 4.1 Code Implementation

### 4.1.1 Map editor

*map_editor.dart*

The entry point for the map editor is a page containing any previously created maps and an option to create a new map. Selecting a previously created map will open a page where the map can be viewed but not edited. Selecting a new map will open the google_maps_viewer widget.

*google_maps_viewer.dart*

The google_maps_viewer uses the google_maps_flutter package, which interacts and controls the google maps api, to display a map. It initialises on the Dublin spire, then moves to the user's current location. The user can translate and zoom the camera by using their finger(s) to drag or pinch the screen. There is a capture button that when pressed will exclude the icon showing the user, convert the current location of the camera into an image and store the Latitude and Longitude of the top right and bottom left corners of the image. This information is passed to the image_editor_page

*image_editor_page.dart*

The image_editor_page.dart utilises our image_editor.dart widget which allows the user to draw on the image and detects the faces drawn by the user. The drawings are saved as a collection of Strokes with each Stroke being a collection of points on the image. The image editor provides multiple tools for the user to use when drawing. The pen tool allows the user to drag their finger across the screen with the points specified by the finger being stored in a stroke. The straight-line tool allows the users to specify a straight line to draw by dragging their finger from the start to the end point. Only these two points are added to this stroke. The eraser tool removes any stroke within a certain distance of the point currently being pressed by the user. The undo button undoes the most recent draw action and the redo button redoes the undo or eraser action. When a new stroke is drawn or removed, the face detector is updated and finds the faces representing the zones on the map image. The user can edit the name or delete any of the faces found by the face detector and can change the map's name. The user can then save the map, which stores the data as a json file and converts the image into a string.

### 4.1.2 Face Detector

*face_detector.dart*

The face detector detects faces with a multi step process as defined in the high-level design sections. First we simplify the raw user input into a more usable format without removing detail. Then we convert the strokes into segments which are used to find the intersections within the graph. Finally we use the segments and intersections to build a planar graph and trace the faces.

```dart
void rebuild(){
  if(rawStrokes.isEmpty){
    graph = null;
    strokes = null;
    segments = null;
    return;
  }

  strokes = strokeBuilder.prepareStrokes(rawStrokes);
  segments = segmentBuilder.buildSegments(strokes!);
  final intersections = segmentBuilder.findIntersections(segments!);
  graph = graphBuilder.buildPlanarGraph(segments!, intersections);
}
```

*face_dectection_stroke.dart*

To prepare the raw strokes provided by the user we run multiple simplification methods.

- We remove any points within a tolerance of each other and snap a stroke's endpoint to its first point if it is within a tolerance.
- Next we remove collinear points, we determine if angle A->B->C is “too straight” by comparing the dot product of the two lines to a tolerance. If the line is “too straight” or if the perpendicular distance from B -> AC is too short we remove B.
- Next we apply the Ramer–Douglas–Peucker algorithm to simplify a polyline. The algorithm selects A and C as the start and end of a line. Next find the point with the farthest perpendicular distance from AC, B. If the distance from B -> AC is outside a tolerance then keep B and repeat the process from A->B and B->C else discard all points in between the start and end.
- We clean the points again then finally we cluster all end points in the graph, where any endpoints from any stroke within a tolerance are clustered and both now point to a new averaged point.

```
List<Stroke> prepareStrokes(List<Stroke> input){
  final simplified = <Stroke>[];

  for(final s in input){
    var points = s.points.map((p) => ui.Offset(p.dx, p.dy)).toList(
    points = simplifyPoints(points);
    points = simplifyColinearPoints(points, 0.98);
    points = rdpSimplify(points);
    points = simplifyPoints(points);

    if(points.length >= 2){
      simplified.add(
        Stroke(
          index: s.index,
          points: points,
          width: s.width,
          color: s.color,
        ),
      );
    }
  }
  snapNearbyEndpoints(simplified);
  return simplified;
}
```

*face_detection_segment.dart*

Next we build the segments and find the intersections. To find the segment to segment intersections, we first create a spatial hash. Using the spatial hash we record in which cells each segment is in, we then compare only cells that contain more than one segment. This dramatically decreases the time taken to process the segment to segment intersections as we no longer have to compare all segments against every other segment. To find the segment to segment intersections, we check if there is an intersection between a1->a2 and b1->b2 then store that intersection for both segments. To find the endpoint intersections we project the endpoint onto each segment and if it's within a tolerance we count it as an intersection. This allows endpoints to be a slight distance away from the segment and still count as an intersection.

```
// spaitial hash so only nearby segments are checked against eachother for intersections
final spaitialHash = <String, List<int>>{};
for(int i = 0; i < segments.length; i++){
  for(final key in cellKeys(segments[i].a, segments[i].b)){
    spaitialHash.putIfAbsent(key, () => []).add(i);
  }
}
```

```
// for each segment
for(int i = 0; i < segments.length; i++){
  final seg = segments[i];
  final candidates = <int>{};
  // find the candidates to be checked
  for(final key in cellKeys(seg.a, seg.b)){
    final ids = spaitialHash[key];
    if(ids != null) candidates.addAll(ids);
  }
  // for each candidate
  for(final j in candidates){
    if(i >= j) continue;
    final other = segments[j];
    final hit = segmentIntersection(seg.a, seg.b, other.a, other.b);
    // if theres an intersection store it
    if(hit != null){
      pushUnique(segmentPoints[seg.ref]!, hit);
      pushUnique(segmentPoints[other.ref]!, hit);
    }
  }
}
```

```
// for each endpoint
for(final ep in endpoints){
  SegmentData? bestSeg;
  ui.Offset? bestProj;
  double bestDistance = double.infinity;
  // for each segment
  for(final seg in segments){
    if(seg.ref == ep.segmentRef){continue;}
    if(pointBuilder.samePoint(ep.point, seg.a) || pointBuilder.samePoint(ep.point, seg.b)){continue;}

    final proj = projectPointOnSegment(ep.point, seg.a, seg.b);
    if(proj == null){continue;}

    final distSquared = distanceSquared(ep.point, proj);
    if(distSquared > endpointToSegmentTolerance * endpointToSegmentTolerance){continue;}
    if(pointBuilder.samePoint(proj, seg.a) || pointBuilder.samePoint(proj, seg.b)){continue;}

    // find the closest segment
    if(distSquared < bestDistance){
      bestDistance = distSquared;
      bestSeg = seg;
      bestProj = proj;
    }
  }
  if(bestSeg != null && bestProj != null){
    pushUnique(segmentPoints[ep.segmentRef]!, bestProj);
    pushUnique(segmentPoints[bestSeg.ref]!, bestProj);
  }
}
```

*face_detection_graph.dart*
We build the graph by sorting all the intersections along a segment and creating the vertices and edges appropriately. First the graph is pruned to remove dead ends, ensuring it is suitable for planar face detection. This is done by iteratively removing vertices with only one connected edge, as these cannot form part of a closed region. Then any bridges from one enclosed region to another are found and marked to be ignored for face detection. This is done by applying Tarjan's bridge finding algorithm.

- This algorithm applies depth-first search to the nodes in a graph, asking for each node how many steps to get to this node and what is the lowest number of steps for a node it can reach without going to its parent node. This identifies bridges which are ignored during face tracing.
-

When the graph has been prepared, we trace the faces using a half edge traversal approach.

- Starting from any directed edge A to B, the algorithm selects the next edge by taking the first edge leaving B that is counter clockwise from A to B, effectively always making the first left turn.
- This process is repeated, moving from edge to edge, until the traversal returns to the starting edge. If a closed loop is formed without revisiting edges prematurely, it defines a valid face.
- Each directed edge is treated independently, meaning A to B and B to A are considered separate half edges. The algorithm continues by selecting another unvisited half edge and repeating the process until all edges have been explored.
- This approach relies on the graph being planar and free of dead ends, as such structures would prevent the formation of valid closed loops.

### 4.1.3 Game Screen

*lobby.dart*
Both *offlinegamelobby.dart* and *onlinegamelobby.dart* builds tabs and delegates them to LobbyLayout, with the only exception being the chat tab that uses onSendChat callback. LobbyLayout acts as a template for the offline and online versions of the game.

Within _buildGameTab, there are subtabs with Rules, Players, and Logs. The offline version adds a fourth "Tree" sub-tab for the host outside of play to visualize the Nearby Connections topology. Above these sub-tabs sits the top section built by _buildTopSection, displaying lobby info, game status, or finished state depending on phase, which uses _buildLobbyTopSection, _buildActiveTopSection, or _buildFinishedTopSection respectively.

```
Widget _buildTopSection(BuildContext context) {
  if (_gameFinished) return _buildFinishedTopSection(context);
  if (_gameStarted) return _buildActiveTopSection(context);
  return _buildLobbyTopSection(context);
}
```

The map tab similarly changes behavior based on game state, before the game starts, it shows the MapSelector for the host to pick a map, and during the active game it shows the GameDisplay with the caught button overlaid for runners.

#### 4.1.4 Offline Connectivity

*offlinegamelobby.dart*

Discovery and advertising is handled using the Nearby class from nearby_connections library by mannprerak2. Advertisers advertise using the lobby name as the name of the endpoint. Once a user has discovered an advertiser and made a request to connect, the request is automatically accepted but users must be manually accepted into the game to gain game information and be included in the active game. This is so both parties can exchange unique userName and displayName.

```
// Exchange user and display names.
_sendBytes(id, Cmd.username(userName, displayName));
```

Nearby Connections API can send messages between connected devices. Messages payloads have a limit of about 32kb (this limit is what is documented, although we found that messages can reach sizes of 700+kb) and are encoded into bytes before being sent. To distinguish between messages sent through Nearby Connections, messages are built as commands and wrapped with formatting. Messages are then parsed upon retrieval.

```
static const String _username = 'CMD:USERNAME:';

// Build
static String username(String userName, String displayName) =>
    '$_username$userName:$displayName';

// Check
static bool isUsername(String m) => m.startsWith(_username);

// Parse
static (String userName, String displayName) parseUsername(String m) {
    final parts = m.substring(_username.length).split(':');
    return (parts[0], parts[1]);
}
```

To send messages of over 32kb, we implemented message chucking. Larger messages such as map files, individual replays and full replay files, are chunked into smaller <32kb messages and rapidly sent to receivers. These chucks are wrapped with a unique ID for the message they are part of and the position (index) the chunk is in in the full message.

```
void _sendChunked(String endpointId, String message) {
  if (message.length <= _maxChunkSize) {
    _sendBytes(endpointId, message);
    return;
  }

  final transferId = _generateTransferId();
  final total = (message.length / _maxChunkSize).ceil();
  _addLog('Sending chunked: ${message.length} bytes in $total chunks');

  for (int i = 0; i < total; i++) {
    final start = i * _maxChunkSize;
    final end = min(start + _maxChunkSize, message.length);
    _sendBytes(endpointId, Cmd.chunk(transferId, i, total, false, message.substring(start, end)));
  }
}
```

The direction in which messages can be sent is variable. A message from the host can be flooded through the network by sending it to every device directly connected to it and making every device send that message to devices connected to themselves, excluding the device it connected to the network through. This is because of the tree shape of the network. Messages can be sent to the host by just continually being relayed upwards through the tree and messages can be sent to specific devices by flooding the network and having the message wrapped with the relay command. The relay command just tells the device that received the message with it to continue flooding the tree if the username specified in the command is not its own.

```
/// Flood a message to all direct connections except [excludeEndpointId].
void _flood(String message, {String? excludeEndpointId}) {
  for (final eid in endpointMap.keys) {
    if (eid != excludeEndpointId) {
      _sendBytes(eid, message);
    }
  }
}
```

The host maintains a list of all players connected and their direct connections in _tree. This is so it can properly maintain a player list, removing all players linked to a pseudo-host no matter the depth. This also allows the host to always find the closest player to it to promote to pseudo-host, which we want to reduce latency and points of failure. If the host connection capacity turns full or a disconnect happens, we check for if there is an advertising pseudo-host in the tree (we know they are advertising if they are not at full capacity) and send a promotion command to the nearest players if there isn't.

```
void _hostHandleDisconnect(String parentName, String childName) {
  _addLog("$childName disconnected from $parentName");

  // Remove the child and its subtree from the tree
  final removed = _removeSubtree(childName);
  // Remove the child from the parent's children list
  _tree[parentName]?.remove(childName);

  // Clear pseudo-host status for all removed nodes
  for (final node in removed) {
    _pseudoHosts.remove(node);
  }
```

#### 4.1.5 Online Connectivity

*game-server/*

Online connectivity is handled through a NodeJS server using sockets. The server listens for events emitted by clients and broadcasts its own. The pattern is simple: clients send updates and receive updates from the server.

- Game objects created in *Game.js* hold all data related to the game and contain methods for the game life.
- Player objects created in *Player.js* hold all data related to players.
- *GameManager.js* contains methods for management of games including creating them and adding players.

Every file in */handlers* are event handlers that the server registers when run.

- *locationHandlers.js* handles the location updates emitted by players.
- *chatHandlers.js* handles the listening and broadcasting of chat messages.
- By far the most important one is gameHandlers.js, which manages all events related to game life.

Locations of Runners are broadcast to Chasers if those Runners are deemed “revealed” from being out of zone. This required player specific broadcasts. A payload is created in *Game.js* for each player and emitted to their specific sockets in a loop using a callback located in *gameHandler.js*.

```
getEachPlayerBroadcast() {
  // Build entries for every player
  const allEntries = {};
  for (const [id, player] of this.players) {
    allEntries[id] = this._buildPlayerEntry(id, player);
  }

  const result = {}; // { socketId: { playerId: entry, ... } }

  for (const [id, player] of this.players) {
    if (!player.connected) continue;

    if (player.role === "runner") {
      continue;
    }

    // Chasers see other chasers and runners not in a zone (excluding themselves)
    const filtered = {};
    for (const [otherId, otherPlayer] of this.players) {
      if (otherId === id) continue; // skip self

      if (otherPlayer.role === "chaser") {
        filtered[otherId] = allEntries[otherId];
      } else if (otherPlayer.role === "runner" && otherPlayer.revealed) {
        filtered[otherId] = allEntries[otherId];
      }
    }

    result[player.socketId] = filtered;
  }

  return result;
}
```

Additionally, players are marked by the server with different statuses. These statuses are reflected for client maps.
"active"  = location received within staleThresholdMs
"stale"   = connected but no recent location (GPS lost, backgrounded, etc.)
"unknown" = never sent a location
"disconnected" = player's socket dropped, may reconnect

*game_socket_service.dart*
This service is a wrapper around the client that handles all communication with the game server for the online mode. The connect() method registers socket listeners which parse incoming data and send it to callbacks in *onlinegamelobby.dart*. There are also a number of methods here for emitting events to the game server.

```
_socket.on('player:joined', (data) {
  onPlayerJoined?.call(Map<String, dynamic>.from(data));
});
```

### 4.1.6 Location Service

*location_service.dart*

This service streams the device's GPS position and processes it for the game.

GPS tracking is done by listening to Geolocator.getPositionStream. When a position arrives it's stored in _latestPosition with a timestamp. If the GPS stream errors out, it retries automatically. A timer fires every sendIntervalMs (5 seconds). Each tick, it checks if a new position from the GPS stream has come in, if it has, it converts the latest GPS coordinates to image-space coordinates using the map's bounds, determines which zone the player is in via map.graph.pointInFace, and decides whether a runner should be revealed (visible to chasers). If there's a socket connection via online mode, it sends the position to the server. It also calls onFrameRecorded so the replay can log the frame.

The service works for both online and offline modes, gameSocketService is nullable, and when it's null the service just skips the server sends but still does all the local processing (zone detection, replay recording, out-of-zone timer).

```
if (_hasSocket && _socketConnected) {
  gameSocketService!.sendLocation(
    x: imageOffset.dx,
    y: imageOffset.dy,
    zone: zone,
    revealed: revealed,
  );
}
```

### 4.1.7 Replays

*replay_file_handler.dart* and *replay.dart*

A ReplayFile stores all data needed to reconstruct a simulation of a completed game. It consists of information about the game and a collection of Replays, one for each player, which contains Frames of the player's locations throughout the game, as well as information about the player like their name. A Frame contains the timestamp, the player's position on the map, and the player's current zone and role.

The host is responsible for building the ReplayFile during the finished phase. The host needs to collect all Replays from individual players and then distribute them. This process is handled automatically whether online or offline but the assembly and distribution can be triggered early if a player drops out.

*replay_viewer.dart*

The replay viewer plays this data back over time to recreate the game visually. It updates at 30 frames per second and at each update determines each player's position for the current playback time. If the playback time falls between two recorded frames, the position is interpolated so that movement appears smooth rather than jumping between recorded points.

The replay viewer also checks the stored zone closing times and updates the map display to show which zones are closed at the current playback time. Selected players are then drawn on top of the map in their current positions allowing the full game state to be replayed visually.

*offlinegamelobby.dart*
The exchange of replays happens in two phases, collection then distribution, and the mechanism for switching between them is the advertising name the host broadcasts with over Nearby Connections. The host uses the name “COLLECT:” followed by the lobby name. Non-hosts discover this, connect, send their replay, get an acknowledgement, and disconnect. Once the host has assembled the final replay file, it starts advertising again with the name “DISTRIBUTE:” followed by the lobby name.

```
Future<void> _startCollecting() async {
  // Add host's own replay
  if (_replay != null) {
    _collectedReplays[userName] = _replay!.toJson();
  }

  setState(() => _collectingReplays = true);

  // Check if host already has all replays
  if (_collectedReplays.length >= _players.length) {
    _assembleAndDistributeOffline();
    return;
  }

  await _startReplayAdvertising('COLLECT:$_lobbyName');
  _addLog('Collecting replays... advertising as COLLECT:$_lobbyName');
}
```

*onlinegamelobby.dart*
The process for gathering and distributing replays is far simpler in the online version of the game because of the server. Individual Replays are saved to the server and sent to the host, then the ReplayFile is saved to the server.

### 4.1.8 File and Session Storage

***map_file_handler.dart* and *replay_file_handler.dart***
Map files and Replay files are stored as encoded json within the application storage. Json provides quick and simple access to the map data and replay data created by the user. When the user creates a map through the Map Editor a json representation is created and saved to the user's device. Replay files are handled the same, being converted to json and stored, with each Replay file containing the map id it was played on. When a replay is downloaded, the corresponding map is also downloaded if it isn't already on the device as it is required to view the replay.

- A map file contains the background image drawn on, the latitude and longitude of the top right and bottom left corners of the image and a collection of faces, edges and vertices.

- A replay file has the id of the map played on, the zone closing times and a collection of replays from each player which contains their location on the map during each location update.
- Both map files and replay files are easily deleted but, because replay files rely on map files, when a map file is deleted we also delete all corresponding replay files for that map.

*offline_game_session.dart* and *gamesession.dart*
Session files are also stored as json within application storage, one for offline and one for online. They store the necessary information to continue a game from an app restart. Session files are continuously written throughout the game; they include the game session file for general details, a game replay file for replay, and a game map file for the map.

- The game session file is updated whenever the game begins, whenever the user enters or exits a zone, when the user's role updates, and when the game finishes. In online mode, when a joiner is accepted into the lobby, on rejoin, when a player joins/reconnects/disconnects, when zones are reordered, after replay assembly, and after sending the assembled replay to the server. The socket service only stores currentGameId and currentPlayerId in memory for reconnection.
- _persistReplay for both offline and online is called every time onFrameRecorded fires from the location service (every 5 seconds) when GPS has a new position. This updates the Replay file.
- The map is saved once it is received and an active game begins.

```dart
Future<void> _persistReplay() async {
  if (_replay == null) return;
  try {
    await GameSession.updateReplay(_replay!.toJson());
  } catch (e) {
    print('Failed to persist replay: $e');
  }
}
```

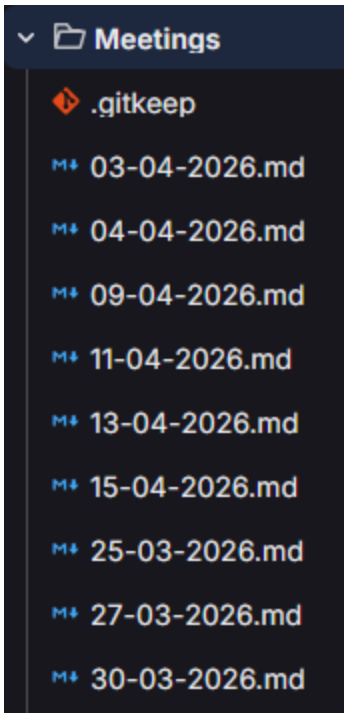
### 4.1.9 Google Maps

We use Google Maps to allow the user to select where they want to make their map. To access the Google Maps API we use the google_maps_flutter package. This package provides a widget which allows easy access to the Google Maps API endpoint and easy API key configuration. Using this we can capture an image of a location and store the latitude and longitude of the top right and bottom left corners. We use this to display the player onto the map. To ensure correct user displaying behaviour we have locked the Google Maps map to only face north. This means there is no rotation allowed but users have access to translation and zoom to find their perfect spot.

## 4.2 Project Implementation

### 4.2.1 Agile

We adopted some elements from Agile work methodology during the development of Tag Battle Royale. Functionality was defined using user stories which were created and refined throughout the development process. A copy of our user stories is held in 2026-csc1097-tag-battle-royale/res/TBR_UserStories.csv
We held meetings, sometimes daily in order to review progress and discuss challenges. A number of meeting files are kept in the repo in the directory named Meetings although this is not a comprehensive list of all the calls and communication made during development. This helped the team stay coordinated and focused throughout development.




### 4.2.2 Continuous Integration (CI)

We implemented continuous integration using GitLab CI pipelines. A custom pipeline was configured to automatically run all automatic tests on each commit and then generate a build of Tag Battle Royale. This reduced the risk of breaking functionality and maintaining stability across the project.

# 5. Problems Solved

### 5.1 Secure API Access

Tag Battle Royale uses a Google Maps API to allow users to select where to play the game. To ensure access with our API is secure we have implemented the following security.

- Obfuscation: The API is only ever stored locally, this ensures that malicious entities cannot easily find our key.
- Identification: Each device using the google_maps_flutter package has a unique hash. Providing this hash and our project name to Google allows only projects with our project name and hashes to access the endpoint. Other people can use the apk file as long as it was generated by one of our members.
- API Key Restriction: The API key has been limited access to only the endpoints used in the project, further securing the key

### 5.2 Displaying Users

During a game of Tag Battle Royale players are drawn onto the map. To achieve this, when the map is captured in the image editor, we also capture the altitude and longitude of the top right and bottom left corners of the map. We then convert the latitude and longitude of the player's position into a point between the two corners. Using this new point we find where on the map the player is and use that image coordinate to draw a player. To ensure this behavior is consistent and correct, we lock the image to only face north.

### 5.3 Face Detection

When a user draws on a map, the application must be able to detect any faces found in those inputs. This proposed a major challenge in two areas.

- Preprocessing: User input is a collection of inconsistent misaligned data. Before we process it and find faces, we must first simplify it and align it. We remove any points we can while preserving the shape and structure defined by the user.
- Processing: The cleaned input must be processed into a planar graph where the face tracing algorithms can apply. To do this we find all intersections, build the graph, remove any dead ends and mark any bridges in the graph. After this has completed we can trace the faces.
- Detection: To find the faces we implemented a half-edge traversal method for detection faces. This algorithm will work correctly only if the previous steps have been completed entirely and correctly.

### 5.4 Offline Connectivity: Scatternet Pseudo-host Allocation

At the early stages of development all players had a list of connected players to a game, but this was insufficient for hosts. Hosts were given a tree to maintain a representation of the actual network shape. It’s important that the host has an accurate reading of the tree so they can identify their closest connections to promote to Pseudo-host. This is done with a breadth-first search over the tree, the node the fewest hops away.

### 5.5 Offline Connectivity: Chunking

Chunking was implemented to overcome the limitation of 32kb messages. It was actually implemented quite later on because we found messages containing our maps were still able to transfer even at a size much larger than 32kb (700+kb), but eventually we started running into issues with bigger maps and replays. There were other approaches we could have taken, Nearby Connections does support three forms of payload: messages, external files, and data streaming.

Sending files through Nearby Connections requires extra effort. The file needs to be saved in external storage outside of the app, which requires permissions and extra work to store and fetch. Data streaming seems like the obvious choice but was not supported by our chosen library. Additionally, the chunking approach actually works better with our Scatternet approach, each chunk can be forwarded immediately upon receipt without any additional overhead. A stream payload would require each relay node to manage an incoming and outgoing stream. Chunking keeps the path simple, every node in the network treats chunks the same way it treats other bytes messages, forwarding them on as they arrive.

### 5.6 Offline Connectivity: Endpoint Names

The Nearby Connections API uses a name parameter on both sides of a connection. Each side sees the other's name. The problem is that this is the only thing each side sees, other than a unique endpoint ID generated upon the start of advertisement or discovery, and a service ID which is a set string identifying the broadcasting app. As a result, the endpointName should be used to encode messages and information about each device. Our implementation repurposes endpointNames in different ways depending on the phase of the game.

During the lobby phase, the host and any pseudo-hosts advertise using the lobby name. Discoverers see this in their endpoint list and display it as a lobby to join. When a pseudo-host takes over advertising from a full node, it advertises with the same lobby name, but the original lobby name is not removed automatically when it disappears. The discovery callback for joiners handles this, when a new endpoint is found with the same name as an existing one, the old entry is removed, so joiners only ever see one entry per lobby.
(discoveredEndpoints.removeWhere((_, ep) => ep.name == name))

On the joiner side, requestConnection passes the joiner's userName as its name, so the advertiser sees the joiner's identity via endpointName. The advertiser's endpointName is the lobby name, while the joiner's endpointName is their user identity. Because of this, the first thing both sides do after connecting is exchange a USERNAME command with their actual userName and displayName, since the advertiser's real identity isn't known from the connection info alone.

After the game ends and everyone has disconnected, the host reuses the endpoint name mechanism to coordinate the replay exchange. Instead of advertising the lobby name, the host advertises as "COLLECT:&_lobbyName" during the collection phase and "DISTRIBUTE:$_lobbyName" during distribution. Non-host players discover these by filtering onEndpointFound for names matching either "COLLECT:$_lobbyName" or

"DISTRIBUTE:$_lobbyName", so they automatically connect to whichever phase the host is currently in. This name itself carries the state of the replay distribution.

**5.7 Resuming Games**

Games can last a long time with players running outdoors with their phones in their pocket, apps can get backgrounded (even if we try to protect against it), be accidentally closed, and devices can restart. Without session persistence, any interruption would lose the entire game and we don't allow new players to join active games so there wouldn't be any way to recover. As a result, session data is continuously written to the device throughout a game as a JSON file. If a session file exists, it's displayed on the main menu.

On resume, both modes reload from session files and reconstruct the game state. The online version tries to reconnect to the server, which sends back the current player list and any missed updates, the saved session provides continuity. The offline version simply resumes standalone play since there's no network to reconnect to.

**5.8 Disposability**

Dispose is a method that is called when an object in the Flutter tree is removed. Disposal is handled at every level to ensure no GPS streams, timers, sockets, or Nearby Connections persist after leaving a game. Closing a Flutter app should destroy all these services as well.

The mounted property is a Flutter flag on State objects that is true when the widget is in the tree and becomes false after dispose is called. Calling setState on an unmounted object throws an error, so every callback that fires asynchronously, such as socket events, is guarded with a mounted check. Even though we null the callback in _unhookCallbacks, there's still a window where they can fire even after dispose, so those mounted checks are necessary to catch those edge cases.

**5.9 Replay Construction**

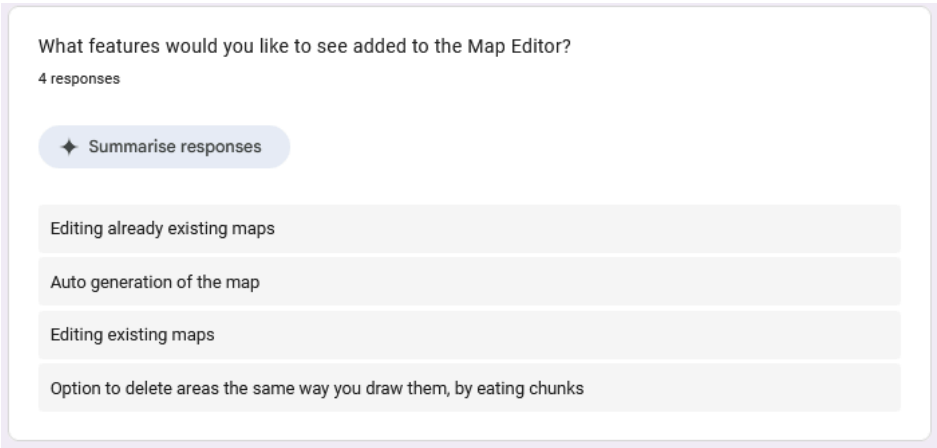
A replay consists of every player's position at any given time during the game. To create this, we have each player store their own location throughout the game along with a timestamp of when it was recorded. At the end of the game, these replays are sent to the host who compiles it into a single replay file. The replay construction system was designed and implemented this way so it could be easily extended from online mode only to offline mode as well.

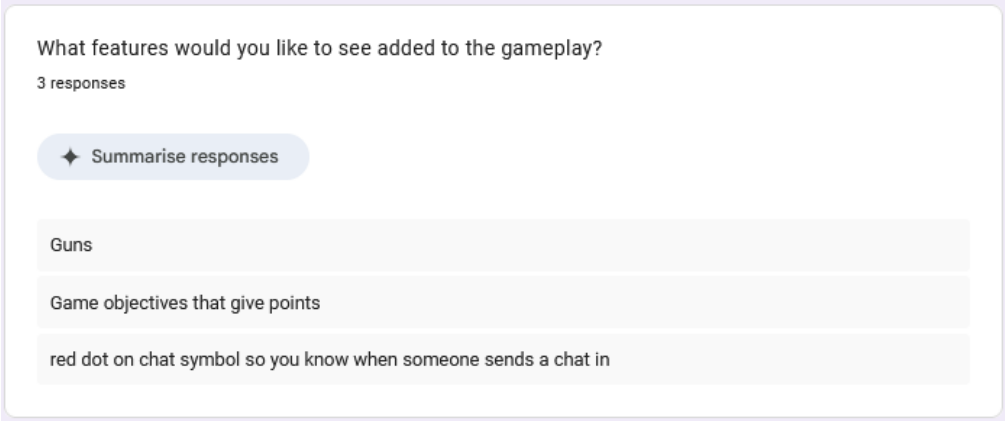
# 6. Future Work

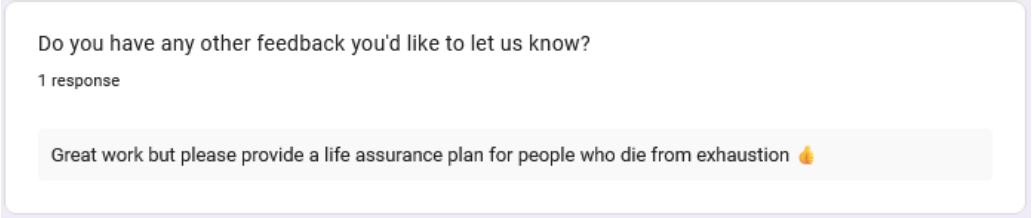
### 6.1 General Feedback

Many of our testers and the people we showed the app to exclaimed that we should have published the app onto app marketplaces for the public. Tag Battle Royale is fully functional and through testing has proved its robustness but before publishing we want to add an additional layer of polish. Through the creation of the app, our team has greatly familiarized ourselves with the Flutter framework and feel we could improve the app UI. Adding more information about saved maps and replays to their respective lists for example, would be helpful. Additionally, further refactoring and new features we weren't able to provide during the timespan of development would provide great benefit. That is not to say we aren't delighted with the end result.

Other feedback included:



### 6.2 Additional Features

- **6.2.1 Automatic Map Features**

  General feedback on the Map Editor was positive, but the ability to create maps and zones automatically without user input would eliminate tedium from playing games. We could utilize the information such as roads, paths, and parks to intuit where the borders of zones should be, relative to the user's position. User input could be taken for the general size of the play space and giving pointers on where the app should be looking at for map/zone generation.

- **6.2.2 Additional Game Modes/Features**

  We realised the sky is the limit when it comes to this. Points that appear on the map that Runners have to run over to collect would boost the number of interactions that happen between Runners and Chasers. A scoring system for collecting said objectives and lasting the longest. Powerups appearing on the map that let Runners see all Chasers or Chasers see all Runners. There can be many variations to the rules, however, simplicity is also a benefit so this would be something we would need to weigh and mull over. Additionally, a completely new method of closing the area of play, like an ever shrinking circle or a king of the hill skill zone system, could be interesting. Due to the way Tag Battle Royale was developed, these features would be extensions of existing functionality and therefore low in difficulty to implement.

# 7. Conclusion

Tag Battle Royale successfully transforms the casual outdoor game into a fully featured Android application capable of supporting large-scale offline and online play. The project tackled several technical challenges, from implementing a Scatternet mesh network using Nearby Connections to building a custom plane face detector for user drawn map zones, in a way that is robust for a game with frequent interruptions.

The map solution required interesting use of ideas like spatial hashing and input processing to efficiently create a planar graph representation of the user's drawing while maintaining full detail. The intricacies of the map editor and map files exposed the usefulness and necessity of thorough testing as another defect always seemed to appear.The offline connectivity solution required creativity to get around the constraints of peer-to-peer device communication, including limited connection counts, payload size restrictions, and the need for a messaging system across a tree topology. On the online side, the server architecture kept things simple but enabled role-specific features like selective location broadcasting and targeted chatrooms. The game was designed to run with the screen off, to recover gracefully from disconnections, and to produce replay files that let players relive their matches.

Feedback from testers confirmed to us that the core gameplay loop is engaging. While there is room for further polish, refactoring, and the additional features, the application in its current state accomplishes the goals set out at the start of the project: Tag Battle Royale is a single, comprehensive platform that replaces the mess of messaging apps and image editors that the original game relied on.

# 8. References

**[1]** R. Spanek, P. Kovar, and P. Pirkl, "The BlueGame Project: Ad-hoc multilayer mobile game with social dimension," presented at the 3rd Annu. CoNEXT Conf., New York, NY, USA, Dec. 2007.
http://conferences.sigcomm.org/co-next/2007/papers/studentabstracts/paper5.pdf

**[2]** B.J. Donegan, D.C. Doolan, S. Tabirca, Mobile Message Passing using a Scatternet Framework, International Journal of Computers, Communications & Control
Vol. III (2008), No. 1, pp. 51-59
https://www.researchgate.net/publication/228388915_Mobile_Message_Passing_using_a_Scatternet_Framework